

Technical Report

Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals. Smart India Hackathon SIH26169. Organisation: Department of Space / Indian Space Research Organisation, Space Applications Centre.

Sections follow the order the problem statement asks for: problem understanding, system architecture, software modules, tracking methods, AI methods, test methodology, performance analysis, future improvements. Figures referenced as `results/...` are produced by the software itself; the numbers in section 7 are copied from `results/batch/<stamp>/envelope.md`.

1. Problem understanding

1.1 Context

FSOC links carry data on a highly directional laser beam. Compared with radio they offer gigabit-to-terabit data rates, need no spectrum licence and are immune to electromagnetic interference, but a small angular pointing error breaks the link. Between moving platforms (satellites, UAVs) the two terminals must therefore run a Pointing, Acquisition and Tracking (PAT) loop. PAT has a coarse stage, which brings the remote terminal's beacon into the camera field of view and holds it there, and a fine stage, which locks the beam precisely. This work addresses the coarse stage.

1.2 The engineering problem in the problem statement

Developing coarse alignment algorithms on hardware needs a camera, a pan-tilt mechanism and optics. The problem statement asks for a software platform that replaces them: a configurable virtual scene with one or more moving beacons, a movable virtual camera with realistic rate limits, a configurable set of disturbances, an automatic detector and tracker that steers the camera, live statistics, and an automatically generated performance log.

1.3 Reading of the specification

- The scene ("screen", $\geq 2000 \times 2000$ px) is what the tracker observes each frame: the problem statement's "simulated video stream". The 640×480 px camera window (4 x 3 deg) is the controlled output and represents where the terminal is pointing. Its centre starts at the screen centre and moves at 5 to 10 deg/s at most.
- With one screen pixel equal to one camera pixel, the instantaneous field of view (IFOV) is $4 \text{ deg} / 640 \text{ px} = 0.00625 \text{ deg} = 22.5 \text{ arcsec}$ per pixel; the 10 px tracking specification is 0.0625 deg; at 5 deg/s and 30 Hz the window moves at most 26.7 px per frame; the maximum platform motion of 20 px per frame therefore consumes 75% of the slew budget.
- Benchmark 2 replaces the simulated scene with the evaluators' .mp4. The tracker and camera control run unchanged on the video frames; the output is the per-frame centroid and the timing and lock metrics.

- Two error terms appear in the specification and are both logged: tracking error (true beacon to window centre) and centroiding error (measured centroid to true centroid).

2. System architecture

DISPLAY (PyQt6)	scene view, camera view with HUD, live plots, parameter panel
INSTRUMENTATION	telemetry bus -> frames.csv, summary.json, report.pdf
SIMULATION LOOP (scenario, seed)	fixed step at the camera update rate; deterministic from
WORLD	scene, beacons, camera window and gimbal, disturbance chain,
renderer	
PERCEPTION	FrameSource (simulator video), classical detector, CNN
detector,	
	sub-pixel centroid, association and identity, IMM estimator
CONTROL	state machine, feedforward + PID rate controller, gimbal limits

One `Simulation` object drives every frame for the GUI, the command line and the tests, so all three execute the same code path. A `FrameSource` abstraction yields frames from the simulator or from a video file; nothing downstream knows which.

Data flow per frame: kinematics advance; the gimbal integrates the previous command; the renderer draws the scene and applies the disturbance chain; the tracker detects, measures, associates and updates the estimator; the state machine chooses what to aim at; the controller emits a rate command clipped by the gimbal model; a telemetry record is written.

3. Software modules

Module	Purpose
<code>engine/config.py</code>	Typed configuration; one field per problem statement parameter row; YAML load and save.
<code>world/scene.py</code>	Backgrounds: starfield, terrain, gradient, flat.
<code>world/targets.py</code>	Beacon kinematics: line, circular, figure of 8, random (Ornstein-Uhlenbeck), spiral, sinusoidal, static.
<code>world/camera.py</code>	Gimbal: pose in degrees, rate and acceleration limits, command latency, IFOV conversions.
<code>world/disturbance.py</code>	Extinction, turbulence (wander, scintillation), PSF blur, platform sway, vibration, Poisson, Gaussian and salt-and-pepper noise, in physical order.
<code>world/renderer.py</code>	Draws the scene with sub-pixel beacon placement; returns ground truth.
<code>perception/detect.py</code>	Classical detector (median, background subtraction, matched filter, adaptive threshold, connected components, shape filters), sub-pixel centroid (centre of gravity plus 2D Gaussian fit), CNN heat-map detector (ONNX).
<code>perception/estimator.py</code>	IMM over constant velocity, constant acceleration and coordinated turn.

Module	Purpose
perception/egomotion.py	Frame-to-frame picture shift by phase correlation.
control/tracker.py	State machine SEARCH, VERIFY, TRACK, COAST, REACQUIRE; gating; appearance signature for identity; adaptive measurement noise.
control/controller.py	Feedforward plus PID rate controller with latency lead, deadband and anti-windup.
engine/simulation.py	The run loop and telemetry record.
engine/metrics.py	Metric definitions and specification pass/fail.
engine/report.py	PDF report.
gui/app.py	Desktop application.
cli.py	run, video, batch, gui commands.

4. Tracking methods

4.1 Detection and centroiding

A 3x3 median (5x5 added automatically under heavy salt and pepper) removes specks. A 31x31 box blur estimates the smooth background, which is subtracted. A Gaussian matched filter at one third of the configured beacon size raises the SNR of an extended spot against point noise and stars. Pixels above mean + k sigma ($k = 4$) are labelled by connected components; blobs are kept if their area, aspect ratio and fill are consistent with a spot. Each blob is scored by SNR, peak brightness and agreement with the expected area of the designated beacon. The leading candidates are refined by an intensity-weighted centre of gravity on a background-subtracted patch followed by a 2D Gaussian least-squares fit, giving centroids to about 0.01 px on clean frames and 0.1 to 0.2 px under heavy noise.

4.2 Estimation

An IMM runs three Kalman filters (constant velocity, constant acceleration, coordinated turn) and blends them by likelihood. Process noise is sized for the accelerations the problem statement produces (beacon paths to $\sim 100 \text{ px/s}^2$, platform sway to $\sim 600 \text{ px/s}^2$). Measurement noise adapts: the zero-mean part of the innovation sequence is treated as vibration and added to the measurement variance, so the estimator smooths jitter instead of chasing it, while a consistent innovation (lag) is not mistaken for noise.

4.3 State machine and identity

SEARCH detects on the whole observed picture. VERIFY requires 3 of 4 consistent frames before declaring lock, so a noise speck is never counted as acquisition. TRACK searches a region around the prediction, gates candidates by Mahalanobis distance and rejects those whose appearance signature (area, peak, PSF width) differs strongly from the beacon being followed, which holds identity through crossings with decoys. COAST propagates the prediction through short dropouts; REACQUIRE

widens the search with the growing uncertainty and falls back to SEARCH if it exceeds the screen. In hard mode (tracker restricted to the window) SEARCH drives an outward spiral.

4.4 Control

The angular error is the IFOV times the pixel offset from the window centre to the target position led by the command latency (velocity and acceleration), with the window's own motion led likewise. The command is feedforward of the target angular rate plus PID on the error, with a 1.5 px deadband and integrator clamping while saturated. The gimbal model applies rate saturation (5 to 10 deg/s), acceleration saturation and a one-frame latency, and reports saturation to the log.

5. AI methods

5.1 Role

The classical detector is fast and accurate in clear conditions but fails in fog, rain, low light and heavy noise, where "the brightest compact blob" is the wrong answer. A small fully convolutional network then provides the measurement. It runs only when the classical confidence is below a floor, and only on a 128 x 128 region around the prediction, so speed and reliability never depend on the model.

5.2 Network and training

Three-level U-Net-style encoder with skip connection, about 0.1 M parameters, input a normalised 128 x 128 patch, output a 64 x 64 heat map with a Gaussian bump on the beacon. Loss: weighted binary cross-entropy. Training data are rendered by the simulator itself under randomised backgrounds, beacon shapes and sizes, decoys, atmosphere presets and noise; the simulator knows the beacon position, so labels are exact and unlimited. The model is exported to ONNX and run with ONNX Runtime on CPU in a few milliseconds. Sub-pixel position comes from a soft-argmax on the heat map followed by the same Gaussian refinement used by the classical path.

6. Test methodology

- Unit tests (`tests/`): IFOV and pose conversions, gimbal saturation and pose limits, determinism and screen bounds of every motion type, centroid accuracy on clean frames, sub-pixel refinement on a synthetic Gaussian, IMM prediction on a circle, closed-loop specification checks on a clear scenario and on a platform-plus-vibration scenario, and a Benchmark 2 path that writes a synthetic video and runs the tracker on it.
- Scenario pack (`configs/scenarios/`): the four mandatory motions in clear conditions, heavy noise, fog, low light, platform sway with vibration, a multi-target stress case, and hard mode.
- Batch envelope: `fsoc-tracker batch --scenario configs/scenarios/*.yaml --seeds 0-N` runs every scenario over N seeds and writes `envelope.md` with mean and worst values.
- Every metric has a printed definition (section 8 of the user manual) so the numbers can be compared with the evaluators' own.

7. Performance analysis

All figures below are measured by the software itself and copied from

results/batch/20260919_015655/envelope.md (five seeds per scenario, 15 s each) and

results/batch_hard/20260919_020453/envelope.md (hard mode, five seeds, 25 s). Tracking error is

the mean distance from the true beacon to the window centre over frames after acquisition;

centroiding error is the mean distance from the measured centroid to the true centroid; lock retention

is the share of frames in TRACK with the estimate within 30 px of the window centre.

Scenario	Acquisition mean / max (s)	Tracking error mean / worst seed (px)	Centroiding error mean (px)	Lock retention, worst seed (%)	FPS, worst seed	Spec rows 16 to 20
clear, line	1.01 / 1.27	7.98 / 8.85	0.01	100.0	187	pass
clear, circular	0.93 / 1.43	6.60 / 7.31	0.01	100.0	182	pass
clear, figure of 8	1.05 / 1.37	7.18 / 7.80	0.01	99.8	178	pass
clear, random walk	1.01 / 1.23	10.13 / 11.48	0.01	98.1	180	mean at the 10 px limit
noise: salt and pepper 10%, Gaussian sigma 20, Poisson	0.95 / 1.10	7.66 / 8.05	0.19	100.0	95	pass
fog	0.91 / 1.43	6.62 / 7.29	0.08	100.0	105	pass
low light	1.07 / 1.43	7.17 / 8.01	0.17	100.0	104	pass
platform sway 12 px/frame + vibration 20 px/frame	0.84 / 0.97	19.89 / 20.11 (14.6 with vibration removed)	0.02	96.5	87	tracking error above 10 px: the truth itself moves 20 px per frame
platform at the PS maximum, 20 + 20 px/frame	0.94 / 1.23	25.30 / 26.25	0.05	79.3	87	fails lock retention: gimbal at 90% of its slew budget
multi-target stress: 3 beacons, haze, noise, sway, vibration	0.95 / 1.27	42.77 / 153.70	26.63	69.2	54	one seed of five swapped to a decoy; the other four hold 13 to 17 px
hard mode, tracker sees only the window	6.93 / 12.07	7.05 / 7.56	0.05	100.0	47	acquisition beyond 2 s: a blind sweep at 5 deg/s needs up to 12 s

Discussion.

- Centroiding accuracy is set by the sub-pixel Gaussian fit and holds at 0.01 px in clear air and 0.2 px under the heaviest noise the problem statement lists. This is the quantity compared in both benchmark rounds.
- Tracking error on smooth paths is 6.6 to 8 px, inside the 10 px specification, with lock at or near 100%. The random walk sits at the limit because its accelerations are unpredictable by construction: the controller can only lead what the estimator can extrapolate.
- Under vibration the raw tracking error exceeds 10 px because the true beacon position jumps up to 20 px every frame; the vibration-removed figure (14.6 px) is the part a rate-limited gimbal can physically follow. Both are printed in every report.
- At the maximum platform motion the gimbal saturates in about a third of frames and lock drops to 79 to 88%. This is a property of the mandated 5 deg/s motor against a 3.75 deg/s disturbance, documented rather than tuned away; raising the limit to the allowed 10 deg/s restores lock.
- Identity among decoys holds in four seeds of five; the failing seed loses the beacon during a sway excursion and re-locks a similar decoy. The designation audit recovers some cases; a stronger appearance model is future work.
- Processing runs at 47 to 190 FPS on a 2000 x 2000 scene on a laptop CPU, against the 20 FPS requirement. Hard mode is slowest because full-window detection runs every frame during the sweep.
- The AI detector reached 8 px validation localisation after retraining and is kept as a gap-filling fallback; it supplied about half the measurements on a real 60 fps phone video and none on the simulated scenes, where the classical detector never loses the beacon.

8. Future improvements

- Reinforcement-learned gain scheduling on top of the classical controller, trained in the same simulator.
- Dual-sensor PAT: a wide field acquisition camera feeding a narrow field tracking camera.
- Learned denoiser for extreme scintillation.
- Beacon modulation with lock-in detection for identity in dense clutter.
- Hardware in the loop: the `FrameSource` and gimbal interfaces already isolate the simulator, so a real camera and pan-tilt unit can be substituted.